Reload or Recompute? A Calibrated Rule for KV-Cache Reuse Across Commodity GPUs

M Mohith
Chinmaya Vishwa Vidyapeeth, Kochi, India
achumohith14@gmail.com
ORCID: 0009-0003-2978-3045

Abstract

A returning conversation can resume from a saved key–value (KV) cache or rebuild its prefix. Faster transfer favors reuse, but faster prefill and cache-continuation overhead can favor recomputation. We study this decision for small open models using a synchronized reference implementation and an independently calibrated rule for resumed-turn time to first token (TTFT). A redesigned T4 study contains 26,040 accepted timings across 7 fp16 models. Predictions frozen before cross-GPU sweeps correctly classify 574/636 resolved decisions on L4, 192/206 on A10 with a PCIe x16 link, and 398/426 on A10G with x8. The interrupted H100 point has 20/21 correct resolved decisions and remains limited. Faster allocations raise the observed simulated-bandwidth break-even boundary at a matched main-model context; L4 also extends the sampled short-context recompute regime. These patterns do not establish exact throughput scaling or reliable long-context extrapolation. Kaggle file reads were physical according to process counters; Modal's were page-cache resident, with physical reads not observed. We release the observations, preregistrations, and numerical claim ledger. This is a measurement study and practical calibration rule, not a new cache system.

Artifacts: github.com/mohi-devhub/reload-or-recompute; frozen release v0.1.0-data.

1 Introduction

A returning conversation can reload saved KV or rebuild its prefix. The relevant delay ends at the next response's first token. Reuse can still lose: it moves bytes, reconstructs views, and runs a different continuation path from fresh prefill.

For small-model operators with limited GPU memory, the useful boundary is local: it changes with the model, allocation, new turn, and CPU path.

[Author placeholder: why I started this study. Replace this paragraph with the motivation and deployment experience behind the experiment.]

Prefix caching, external KV stores, and compute-versus-load decisions are established serving techniques. Cake overlaps computation and loading [5]; py-kvcache already characterizes hardware-dependent reuse frontiers [6]. We ask a narrower question: how far does a simple, independently calibrated, whole-prefix TTFT rule transfer across the measured commodity allocations, and where does it fail? We do not claim novelty for KV reuse, offload, a bandwidth threshold, or choosing between compute and load.

Contributions. We make four contributions. First, we release a controlled reference-path measurement of returning-turn KV reuse on older and commodity GPUs, with separate CPU-memory and local-file paths. Second, we state a practical break-even bandwidth rule that includes the resident new-turn baseline and measured CPU and transfer costs. Third, we freeze predictions from short calibration before testing new GPU allocations, and report decision errors, unresolved cells, and the partial H100 run without refitting. Fourth, we identify limits that matter to operators: short-context overhead, long-context extrapolation error, file-path observability, and allocation differences between A10 and A10G.

The single-request Hugging Face reference path uses pre-existing caches. It does not measure admission, reuse probabilities, serving throughput, or overlap-capable engines. Persistence admission additionally depends on cache creation, write cost, capacity, and return probability. Our rule is conditional on a saved cache already existing.

2 The TTFT-adjusted decision rule

Let L be the cached prefix length, N the new-turn length, and $S(L)$ the KV payload in bytes. Let $T_{\text{recompute}}(L, N)$ measure fresh prefill of prefix plus new turn through first-token selection. Let $T_{\text{GPU hit}}(L, N)$ measure continuation when the prefix cache is already resident. For a chosen transfer leg with effective bandwidth B, use

$$\widehat{T}_{\text{reload}}(L, N; B) = \widehat{T}_{\text{GPU hit}}(L, N) + \widehat{h} + \frac{S(L)}{B} + \widehat{T}_{\text{other}}(L, N). \tag{1}$$

Here h is fixed overhead, and T_{other} contains measured CPU or transfer legs not represented by S/B. For a file path this may include read and deserialization work, with H2D transfer represented either by S/B or by another measured leg. Costs must be counted once. Direct `torch.load` and staged byte loading have different CPU paths; their cost models must follow those paths.

Writing $D = \widehat{T}_{\text{recompute}} - \widehat{T}_{\text{GPU hit}} - \widehat{h} - \widehat{T}_{\text{other}}$, the break-even bandwidth is

$$B^*(L, N) = \frac{S(L)}{\widehat{T}_{\text{recompute}}(L, N) - \widehat{T}_{\text{GPU hit}}(L, N) - \widehat{h} - \widehat{T}_{\text{other}}(L, N)}, \qquad D > 0. \tag{2}$$

For a positive denominator, reload is predicted faster above B^*. If the denominator is nonpositive, the additive model predicts no finite bandwidth at which that path wins. This is a model decision, not proof that every implementation must lose. Near a zero denominator, small baseline errors cause large threshold errors; we retain uncertainty rather than force a finite crossing.

A prefix-only approximation compares S/B with prefix prefill alone:

$$B^*_{\text{prefix}}(L) = \frac{S(L)}{T_{\text{prefill}}(L)} = k_{\text{KV}}\, r_{\text{prefill}}(L), \tag{3}$$

where k_{KV} is bytes per token and r_{prefill} is measured prefix tokens per second. This approximation helps explain the bandwidth axis. It does not account for the resumed-turn baseline, allocations, tensor views, or CPU work. In particular, reusing a short prefix can leave too little saved time to pay those costs.

The rule has two tests. The T4 redesign evaluates descriptive agreement with measured same-session recompute and resident-hit baselines. Cross-GPU transfer instead predicts those baselines and transfer legs from calibration, freezes the predictions, and scores later sweeps. The latter additionally tests baseline prediction and allocation stability.

3 Setup and measurement protocol

3.1 Three phases and observed hardware

The pilot tests the main model on T4 at prefixes 512, 2,048, and 8,192, with 32- and 256-token turns. The pilot has 12 measured repeats per cell; the later sweeps use 10. Reload-path medians beat recomputation throughout that pilot grid, so it did not locate a crossover length. The next phase changes the axis to simulated bandwidth and includes short prefixes, multiple model families, separate file legs, and repeated sessions. It contains 26,040 accepted full-sweep timings, 0 invalid full-sweep timings, and 10 sweep sessions across 7 models.

The transfer phase uses L4 and Modal's Ampere allocation tier. The actual boards differ: the main A10 session has an active PCIe generation-4 x16 link, while the main A10G session and the Ampere control-model session have generation-4 x8 links. We report them separately. Each has only one main-model session, so their difference is not within-board session replication. L4 has 2 main-model sessions; T4 has 3. H100 contributes an interrupted optional allocation with only complete recovered cells scored.

All measured weights and caches use fp16. The transfer runs pin Torch `2.11.0+cu130` and Transformers `4.51.3` to the T4 redesign versions. Python patch versions and GPU drivers differ between platforms. The efficient/math SDPA selection is fixed, without FlashAttention. Source hashes protect the timing, forward, continuation, transfer, and file-read routines against changes from the T4 reference. Platform wrappers change allocation checks, scheduling, and persistence, not those protected routines.

3.2 Models, workloads, and coverage

Table 1 lists the measured checkpoints and verified cache sizes. The formula for fp16 KV bytes per token counts keys and values, layers, KV heads, head dimension, and bytes per element. Configuration sizes are checked against the tensors.

Checkpoint	Layout	KV bytes/token
`Qwen/Qwen2.5-0.5B`	GQA	12,288
`Qwen/Qwen2.5-1.5B`	GQA	28,672
`Qwen/Qwen2.5-3B`	GQA	36,864
`EleutherAI/pythia-1.4b`	MHA	196,608
`facebook/opt-1.3b`	MHA	196,608
`microsoft/phi-2`	MHA	327,680
`unsloth/Llama-3.2-3B-Instruct`	GQA	114,688

Table 1: Released fp16 models and verified KV payload. The Llama checkpoint is the ungated mirror. These are different models, not a controlled attention-layout intervention.

The main model uses a frozen synthetic UltraChat fixture, with distinct conversations stitched for long contexts. Repeated text supplies separate auxiliary-model controls. Neither workload establishes quality or a natural return distribution.

The transfer main-model grid has prefixes 16, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384, with new turns 32 and 256. Controls use a 32-token turn and a smaller repeated-text grid, including native-supported limits where feasible. A10 has main-model measurements only; A10G and L4 also have the controls. Context and memory exclusions remain explicit. Appendix A gives the full completed coverage, including T4-specific reductions and the incomplete H100 subset. We do not replace missing cells with predicted measurements.

3.3 Paths and simulated bandwidth

The reference measures fresh recomputation, a GPU-resident hit, pinned CPU reload, pageable CPU reload, and local file API paths. The raw staged path reads bytes, constructs a CPU tensor view, and copies to GPU. The direct torch path includes deserialization in `torch.load`. Independent microbenchmarks measure H2D movement and file CPU legs without the new turn. Cache creation, CPU offload, packing, pinning, file writes, tokenization, input-ID placement, request queueing, and network transport are outside returning-turn TTFT.

For the *simulated-bandwidth* paths, a packed fp16 buffer is copied in chunks. Each chunk is synchronized, and a cumulative rate limiter waits for its byte deadline. Targets are 50, 100, 200, 400, 800, 1,600 MB/s, plus an unthrottled control. MB/s uses decimal bytes. We record target and achieved rates separately. Buffer allocation participates in the pacing budget, and tensor-view work remains inside TTFT; therefore adding an unthrottled overhead intercept is an approximation, especially for short prefixes. This simulates serialized blocking transfer, not physical devices, network latency distributions, or asynchronous pipelines. The unthrottled pinned path is the real pinned-memory control and is counted once.

3.4 Timing, calibration, and uncertainty

Full redesign and transfer sweeps use 10 measured randomized rounds after warmup. Conditions, including prefix-only prefill, are shuffled within each round. CUDA synchronization and events accompany synchronized wall-clock timing; events can include device idle time during host pacing. The endpoint is first-token selection available to the host. Restored-cache continuation consumes the transferred tensors and is checked against the resident-cache continuation and full-recompute first token. This guards against timing a copy that is never used.

Reported within-session intervals use a percentile bootstrap with 10,000 resamples and 95% coverage. Decision scoring uses a paired-round bootstrap interval for reload-minus-recompute TTFT. A reload is resolved faster when that interval excludes zero below, and recomputation is resolved faster when it excludes zero above. Otherwise the cell is unresolved and excluded from the correct/resolved decision denominator. Median absolute TTFT error includes complete baseline and reload cells. Related cells share models, contexts, calibration, and sessions; decision counts are descriptive, not independent Bernoulli trials.

Before each new GPU sweep, calibration measures prefix-only, recompute, and resident-hit baselines at a few lengths, plus independent transfer and file legs. It uses 3 baseline rounds and 5 microbenchmark repeats. Nonnegative quadratic time models predict prefill and recompute; resident-hit and byte-cost legs use nonnegative affine fits. Predictions for every planned cell are committed before its sweep begins. Sweep observations never update coefficients. We distinguish calibration anchors, interpolation, and extrapolation in the released scorecards.

Separate session medians are retained. Session spread is the range of those medians divided by their median, not a population interval. Telemetry samples SM and memory clocks, temperature, power, and event reasons at about one sample per second; Modal also records PCIe generation and width. Sampling can miss brief trials, and associations with clocks do not establish a latency cause.

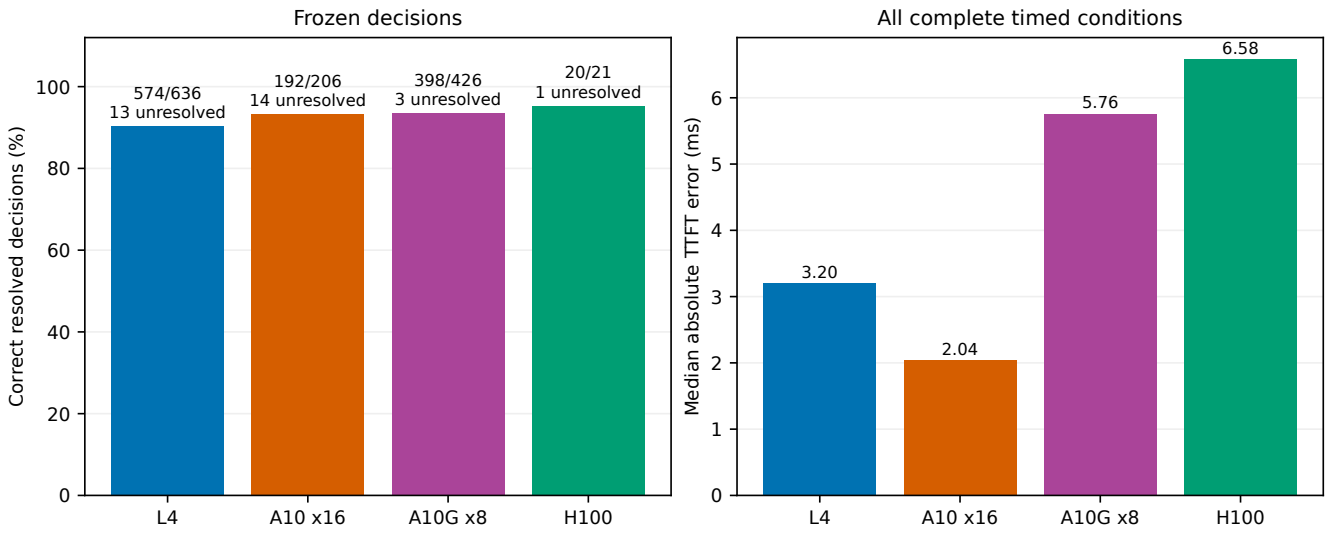


Figure 1: Frozen predictions by observed board. Accuracy excludes paired-interval-unresolved decisions. Median absolute TTFT error includes all complete predicted timed conditions. A10 x16 covers the main model; A10G x8 also covers controls. H100 is partial. Counts are correlated cells, not independent trials, and are not directly comparable under equal coverage.

3.5 File-path observability

Kaggle file reads were physical according to process I/O counters: 7,762/7,762 recorded file observations have positive `physical_read_bytes`, ranging from 196,608 to 1,879,052,288 bytes. These counts include calibration and file microbenchmarks. The counters support reads beyond kernel page-cache hits; they are not physical-device traces and do not identify underlying media or host/storage caches.

Modal's were page-cache resident: 0/4,575 file observations have positive physical-read counters. We use the terminology *local file API path (page-cache resident; physical reads not observed)*. Residency is the operational interpretation of those zero counters and the virtualized filesystem, not proof about every hidden cache layer. Global cache dropping was denied on Kaggle, and per-file eviction advice is advisory on both platforms. We did not perform a cached-versus-uncached ablation. The platforms therefore do not provide equivalent physical-file-read conditions.

4 Results

4.1 Frozen calibration transfers to new GPU allocations

The headline result is supported transfer of the frozen decision rule, with explicit coverage limits. Figure 1 shows 574/636 correct resolved decisions on L4, 192/206 on A10 x16, and 398/426 on A10G x8. Their median absolute TTFT errors are 3.203, 2.040, and 5.758 ms. The unresolved counts are 13, 14, and 3, respectively. H100 has 20/21 correct resolved decisions and 1 unresolved decision among its complete cells.

Calibration observes baselines and independent cost legs, not real reload TTFT or simulated-bandwidth TTFT. Later combined paths test their composition. Baseline-fit error, overhead approximation, and allocation changes remain mixed; a high aggregate count does not justify unmeasured contexts. Appendix B retains per-board, per-region, and per-session scores instead of pooling A10 with A10G.

4.2 Faster allocations raise the bandwidth boundary

At main-model prefix 1,024 and new turn 32, the T4 sessions bracket the simulated-bandwidth crossover between target rates of 100–200 or 200–400 MB/s. A10 x16 brackets it between 800 and 1,600 MB/s; A10G x8 between 400 and 800 MB/s. L4 also crosses between 800 and 1,600 MB/s, although one session has an unresolved endpoint interval. Figure 2 plots achieved-rate brackets at matched lengths, preserving individual sessions and censoring.

This supports a higher bandwidth requirement on faster allocations, while CPU paths, drivers, and links prevent an isolated causal GPU comparison. H100's complete matched short-context point remains recompute-favored throughout the tested simulated rates and unthrottled control. A censored boundary is not an observed finite threshold.

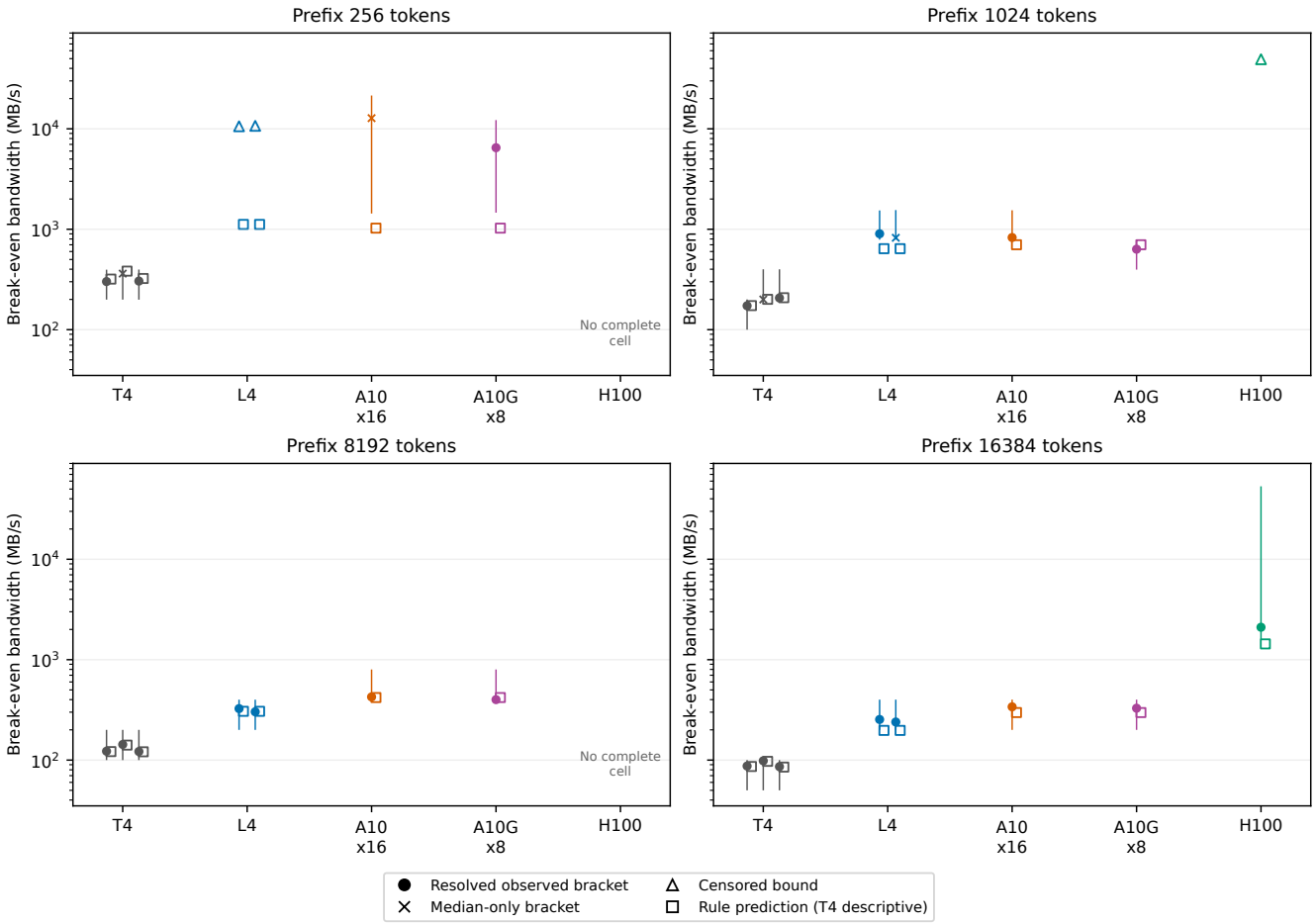


Figure 2: Main-model UltraChat simulated-bandwidth crossovers at matched prefixes, with a 32-token turn. Bars span achieved-rate endpoints, not threshold confidence intervals. Circles require resolved endpoints; crosses have an unresolved endpoint. Triangles are censored median bounds; squares show the rule, descriptive on T4 and frozen on new hardware. Each session remains separate. Missing H100 panels are incomplete or unmeasured. A10 x16 and A10G x8 are distinct allocations.

4.3 Returning-turn and fixed costs change the answer

The T4 redesign's TTFT-adjusted rule agrees with 1,237/1,258 resolved simulated-rate cells, compared with 1,017/1,258 for the prefix-only rule. These are descriptive counts using same-session measured baselines. They support the missing-cost correction, but must not be presented as the frozen cross-GPU score.

A concrete example is Qwen2.5-0.5B at prefix 512 and turn 32. The prefix-only threshold is 226.626 MB/s, while the adjusted threshold is 2,442.265 MB/s. The observed simulated-bandwidth crossing lies between the highest paced rate and the unthrottled control. The small cache payload does not guarantee a cheap returning-turn path: the time saved by reuse must pay for continuation and fixed work.

Fixed first-session calibration also predicts later T4 main-model sessions with 148/149 and 146/150 correct resolved decisions. This is a separate cross-session check on the same GPU type and input, not evidence about an unseen GPU type. Together these results explain why the relevant quantity is the TTFT *saving budget* in Equation 2, rather than prefix FLOPs or KV size alone.

4.4 The sampled short-context regime widens on L4

For a 32-token turn and common prefixes 16, 64, 128, 256, both L4 sessions resolve in favor of recomputation throughout. T4 favors reload at prefix 256 in each session; its smaller-prefix decisions include recompute wins and unresolved cells. Figure 3 shows the separate sessions and turn lengths.

L4 therefore extends the sampled recompute-favored region. This does not hold identically for every faster board: A10 and A10G differ at the boundary, and turn length changes the saving budget. Fast H2D movement cannot remove dispatch and continuation costs.

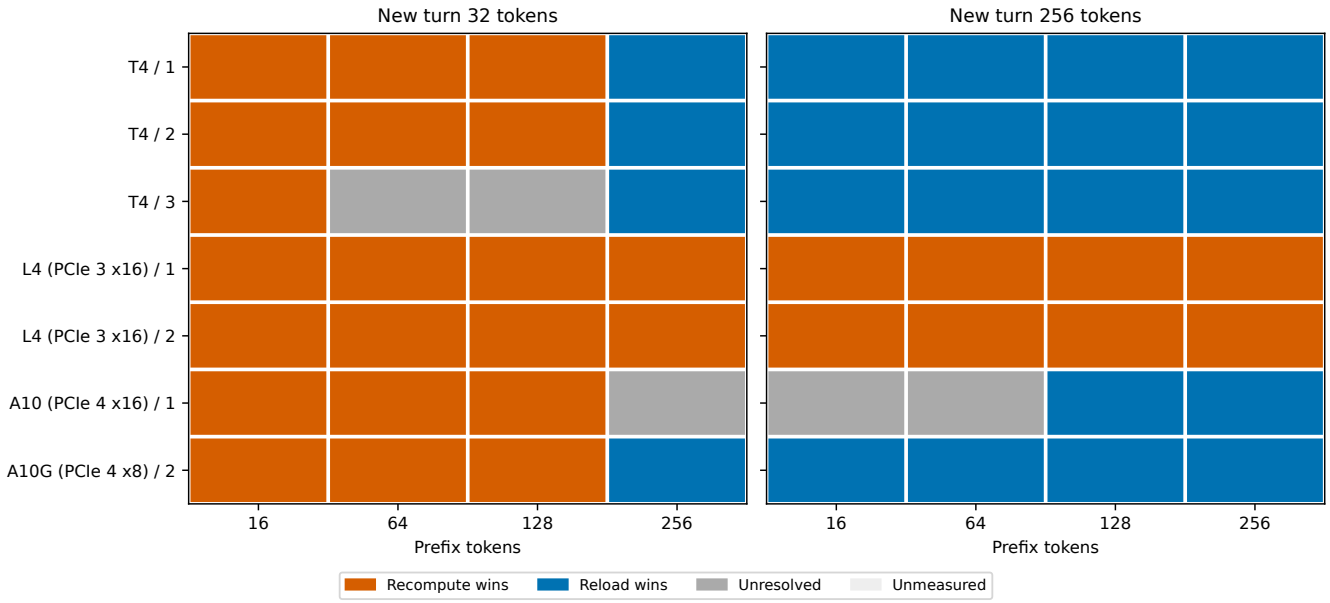


Figure 3: Main-model UltraChat decisions for fresh recomputation versus the real unthrottled chunked pinned-copy path. Rows are distinct sessions. Colors use the archived paired-round bootstrap interval; unresolved cells remain visible. The panels separate the two new-turn lengths. A10 x16 and A10G x8 are different allocations, not within-board replicas. This sparse grid does not identify a continuous crossover length.

4.5 Supporting model and platform observations

At matched repeated-text prefix 512 and turn 32 on T4, Pythia's achieved simulated crossover bracket is $[798.9, 1597.2]$ MB/s, with an unresolved endpoint, versus $[199.5, 398.4]$ for Qwen2.5-1.5B. Phi-2's bracket is $[799.2, 1598.4]$, versus $[99.9, 199.7]$ for Qwen2.5-3B. These selected MHA models need higher simulated bandwidth than the selected GQA models. Different parameter counts, prefill kernels, and widths confound an architectural interpretation; we do not estimate a causal effect of GQA.

Including deserialization reduces the T4 torch-file rule's median absolute relative residual from 11.78% to 3.097%. The raw-format residual changes only from 3.107% to 3.086%. The correction matters to the relevant CPU path rather than improving every format equally. This residual analysis uses independent file legs and same-session baselines; it is not a new frozen-transfer test.

Recompute session spread also matters. Median relative ranges across lengths are 12.341% and 14.847% on T4 for the short and long turn, versus 4.956% and 4.417% on L4. Narrow within-session intervals do not remove allocation variation. The earlier Ampere service-tier ranges of 23.611% and 22.664% compare A10 x16 against A10G x8. Those board/link allocation differences are the likely source of the service-tier spread, but causality is unproved; neither board has repeated main-model sessions. The slower session does not have lower sampled SM clocks, so clock frequency alone does not explain the recompute difference.

5 What does not hold

Long-context extrapolation. The low median TTFT errors conceal large native-limit failures. For Qwen2.5-3B at prefix 32,736 with turn 32, A10G predicts 8,041.096 ms recompute but observes 13,914.555 ms, an absolute error of 5,873.459 ms. L4 predicts 13,088.360 ms and observes 18,022.902 ms, an error of 4,934.542 ms. A short nonnegative quadratic calibration is therefore not a reliable native-context TTFT predictor for these observations. High decision agreement at other lengths cannot justify that extrapolation.

Exact proportional throughput scaling. Figure 4 relates the observed simulated-bandwidth boundary to prefix throughput. The frozen throughput-ratio prediction falls inside at least one matched T4 crossover-ratio bracket for 25/27 ratio-resolved L4 shapes, 11/12 A10 shapes, and 20/24 A10G shapes. The partial H100 comparison is 0/1. These are broad-bracket compatibility counts, not tests proving equality. Censored or unresolved thresholds do not supply exact ratios. The adjusted denominator also changes with continuation and fixed costs. Dividing two prefix-only formula estimates would merely restate an identity, not validate hardware transfer.

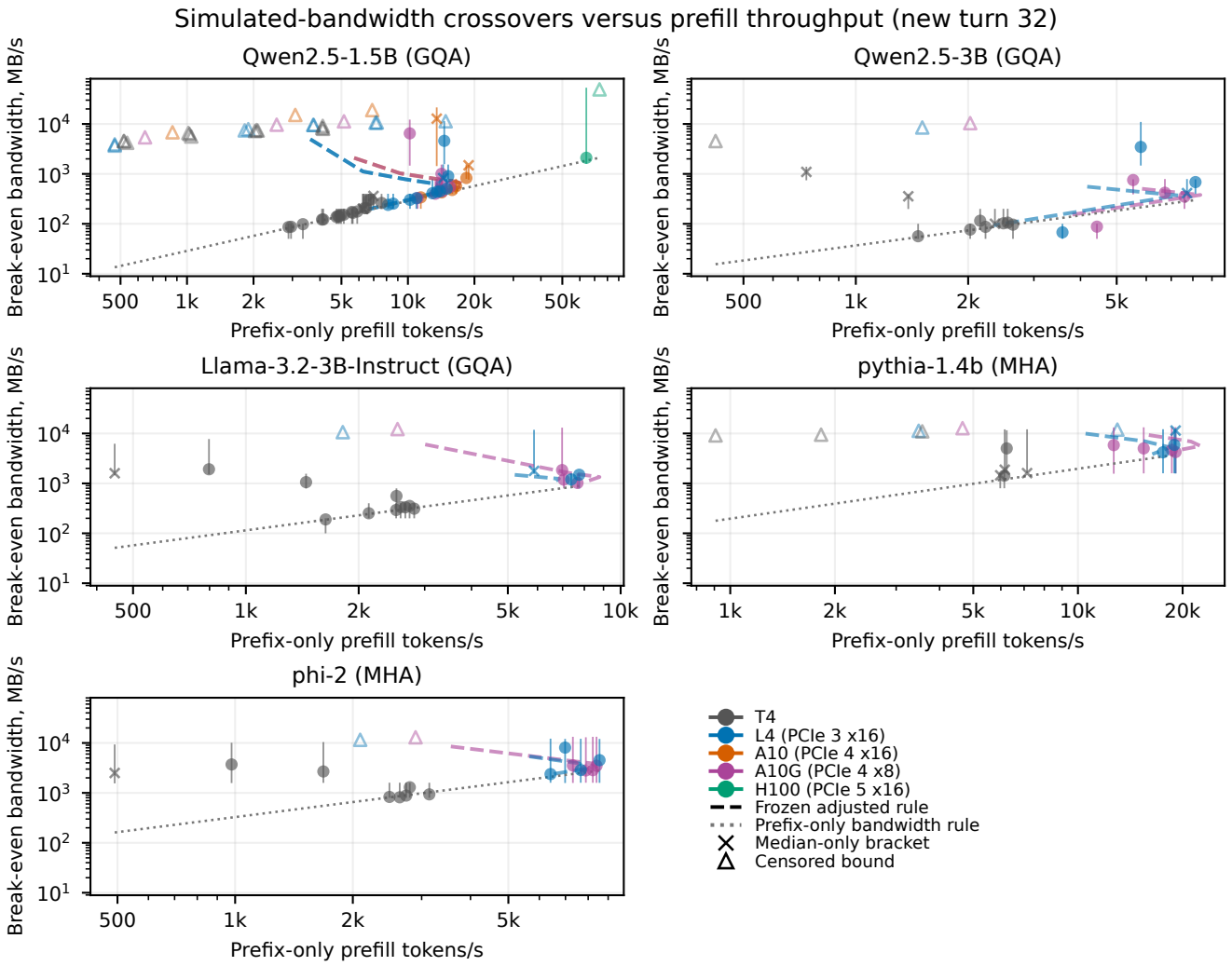


Figure 4: Simulated-bandwidth crossovers versus measured prefill throughput, new turn 32, five models. T4 is descriptive; L4, A10 x16, A10G x8, and partial H100 predictions are frozen. Bars are achieved-rate brackets, not threshold intervals; crosses have an unresolved endpoint and triangles are censored. Dashed curves are the adjusted rule; dotted lines are KV bytes/token times throughput. Interpolated points are not measured thresholds. Model comparisons are confounded, and H100 has no MHA sweep.

Uniform MHA file outcomes. On repeated-text, matched prefixes 64, 256, 512, 1,024 and a 32-token turn, Pythia's direct-file path has no resolved reload wins on T4, but wins at a matched prefix on A10G. Phi-2 already has direct-file wins on T4, so a blanket assertion that all MHA file paths lose there is false. Both measured MHA models lose on both L4 file paths. A10 x16 has no MHA sweep. These outcomes do not transfer uniformly, and Section 3.5's different read conditions prevent attributing the change to GPU speed alone.

The pilot anomaly remains unexplained. The pilot's prefix-only median was slower than prefix-plus-turn at the long pilot context. None of the 4 phase-two anomaly comparisons has a positive interval lower endpoint for that reversal. The historical cause was not established. Non-reproduction and randomized ordering improve interpretation, but do not retroactively identify an implementation, clock, or thermal cause.

6 Practical guidance

A one-minute calibration checklist, not a runtime guarantee. After loading the model, check a short, a representative, and a long context you actually intend to serve. Warm them, then measure synchronized fresh recompute and resident-hit TTFT for the intended turn length. Measure pinned and pageable H2D separately; for a local file API path also time its CPU read/deserialization work and inspect physical-read counters. Use the complete reload estimate and Equation 2; do not subtract the same leg twice. Validate a predicted win with paired reload/recompute rounds before enabling it. Recalibrate when the board, link, model, dtype, turn size, or file path changes.
The released main-model calibration function wall times were 42.5 seconds on A10 and 63.4 seconds on L4. They exclude provider initialization and do not guarantee this checklist's duration. Do not trust a near-zero saving budget, an unmeasured long-context extrapolation, a bracket crossing zero, or a file path with unknown cache semantics. Extend calibration or retain an unresolved decision when the evidence is close.

The checklist is proposed guidance, not an evaluated deployment. Engine builders must remeasure their critical path: preallocation, layout, asynchronous copies, and scheduling change the model terms. Our whole-prefix estimate does not bound an optimized overlap scheduler. Persistence admission also needs write cost, expected return, and memory pressure.

7 Related work

LMCache provides external KV storage, movement, and engine connectors [7]. vLLM reuses matching prefixes and supports KV offloading [2, 3]. SGLang HiCache adds GPU, host, and external-storage policies and prefetch [1]. These systems already serve repeated-prefix and multi-turn use. Our reference loop does not reproduce their production batching and overlap.

CacheGen compresses and streams KV state with bandwidth-adaptive choices [8]. CacheBlend reuses cached chunks and selectively recomputes states to recover cross-chunk attention [11]. We use uncompressed KV for an identical prefix. We therefore measure neither compression/quality trade-offs nor approximate cached-knowledge fusion.

Cake computes from the beginning of a prefix while loading from its end, overlapping compute and load [5]. Our B^* describes the whole-prefix boundary around which such methods split work; it does not predict their optimized overlap time. We did not test overlap or replace Cake's scheduler.

The closest characterization is py-kvcache [6]. It compares external KV reuse and recomputation across hardware and includes a break-even criterion, asynchronous movement, and scheduler-aware preloading. This already establishes the central trade-off and the possibility that weaker hardware benefits while faster hardware does not. We add a bounded reference-path replication with commodity allocations, short contexts, separate file legs and sessions, and frozen prediction checks.

He et al. distinguish buffered reads from physical-read evidence in observational deployment snapshots [4]. KVServe adapts compression to service conditions [9]; ShadowServe addresses fetching interference [10]. These reinforce costs beyond bandwidth. We adopt no external performance numbers or causal interpretation of host-file differences.

8 Limitations and conclusion

The single-request fp16 reference path establishes neither production-engine performance, multi-request throughput, quantization, bf16, P100 transfer, nor application quality. Production attempts yielded no successful performance observation. Synthetic or repeated text and stitched long inputs do not model a natural return distribution.

Coverage is unequal across boards and model families. H100 has only 26 complete timing cells and 52 unfinished registered cells; only main-model prefixes 1,024 and 16,384 are complete, and its Pythia sweep did not start. Partial records remain in the release but are not scored. One H100 allocation cannot establish a general datacenter-GPU or MHA conclusion. Native and memory limits censor other contexts, and the sparse simulated-rate grid often gives a wide bracket to the unthrottled endpoint.

Frozen-fit uncertainty is not propagated. Within-session intervals and few sessions do not describe a deployment population. File counters identify neither media nor all host caches. Serialized copies do not predict overlap or serving interference. Model differences confound architecture; allocation differences confound GPU causality.

Within the measured scope, local calibration predicts most resolved decisions. Operators should measure complete returning-turn paths, retain unresolved outcomes, and recalibrate after changes.

References

[1] SGLang: HiCache System Design and Optimization. Official project documentation. No individual authors or publication date supplied; accessed November 3, 2026. https://docs.sglang.io/docs/advanced_features/hicache_design.

[2] vLLM: Automatic Prefix Caching. Official project documentation. No individual authors or publication date supplied; accessed November 3, 2026. https://raw.githubusercontent.com/vllm-project/vllm/main/docs/features/automatic_prefix_caching.md.

[3] vLLM: KV Offloading Usage Guide. Official project documentation. No individual authors or publication date supplied; accessed November 3, 2026. https://raw.githubusercontent.com/vllm-project/vllm/main/docs/features/kv_offloading_usage.md.

[4] Ying He, Dingsen Shi, Yanbo Dai, Donger Chen, Harry Fu, Chris Tsu, Wang Feng, and Song Fu. Characterizing Predictability-Latency Trade-offs of KV-Cache SSD Offloading in LMCache for LLM Serving Systems. arXiv:2610.21249v1, 2026. https://arxiv.org/abs/2610.21249v1.

[5] Shuowei Jin, Xueshen Liu, Qingzhao Zhang, and Z. Morley Mao. Compute Or Load KV Cache? Why Not Both? arXiv:2410.03065v2, 2025. https://arxiv.org/abs/2410.03065v2.

[6] Joseph Kanichai, Tiziano De Matteis, and Animesh Trivedi. Building py-kvcache: A Performance Characterization of External KV Caching for vLLM with NVMe SSDs. arXiv:2609.11744v1, 2026. https://arxiv.org/abs/2609.11744v1.

[7] Yuhan Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. arXiv:2510.09665v2, 2025. https://arxiv.org/abs/2510.09665v2.

[8] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. arXiv:2310.07240v6, 2024. https://arxiv.org/abs/2310.07240v6.

[9] Zedong Liu, Xinyang Ma, Dejun Luo, Hairui Zhao, Bing Lu, Wenjing Huang, Yida Gu, Xingchen Liu, Zheng Wei, Jinyang Liu, Dingwen Tao, and Guangming Tan. KVServe: Service-Aware KV Cache Compression for Communication-Efficient Disaggregated LLM Serving. arXiv:2605.13734v1, 2026. https://arxiv.org/abs/2605.13734v1.

[10] Xingyu Xiang, Raj Joshi, Yuhan Liu, Jiayi Yao, Chenxingyu Zhao, Junchen Jiang, Yang Zhou, Eddie Kohler, and Minlan Yu. ShadowServe: Interference-Free KV Cache Fetching for Distributed Prefix Caching. arXiv:2509.16857v1, 2025. https://arxiv.org/abs/2509.16857v1.

[11] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. arXiv:2405.16444v3, 2025. https://arxiv.org/abs/2405.16444v3.

A Full completed coverage

The tables are generated from the immutable release, not a new analysis fit. Full checkpoint identifiers and KV sizes appear in Table 1. Phase labels distinguish the pilot, T4 redesign, and transfer. Native-context and budget exclusions are retained; no table entry implies an unmeasured comparison. A10 is PCIe x16, A10G is x8, and their rows are separate.

Table 2: Complete measured coverage. U is UltraChat; R is repeated text. Sessions counts separate allocations, not repeat rounds. Calibration and incomplete H100 cells are excluded.

Phase/board	Model	Text	Turn	Prefix tokens	Sessions
T4	pythia-1.4b	R	32	16, 32, 64, 128, 256, 512, 1,024, 1,792	1
T4	pythia-1.4b	R	256	16, 32, 64, 128, 256, 512, 1,024, 1,792	1
T4	Qwen 0.5B	R	32	16, 32, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192	1
T4	Qwen 0.5B	R	256	16, 32, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192	1
T4	Qwen 1.5B	R	32	16, 32, 64, 128, 256, 512, 1,024, 1,792, 2,048, 8,192	1
T4	Qwen 1.5B	U	32	16, 32, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	3
T4	Qwen 1.5B	U	256	16, 32, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	3
T4	Qwen 3B	R	32	16, 32, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	1
T4	Qwen 3B	R	256	16, 32, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	1
T4	opt-1.3b	R	32	16, 32, 64, 128, 256, 512, 1,024	1
T4	opt-1.3b	R	256	16, 32, 64, 128, 256, 512, 1,024	1
T4	phi-2	R	32	16, 32, 64, 128, 256, 512, 1,024, 1,792	1
T4	phi-2	R	256	16, 32, 64, 128, 256, 512, 1,024, 1,792	1
T4	Llama 3.2-3B	R	32	16, 32, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	1
T4	Llama 3.2-3B	R	256	16, 32, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	1
A10	Qwen 1.5B	U	32	16, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	1
A10	Qwen 1.5B	U	256	16, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	1
A10G	pythia-1.4b	R	32	64, 256, 512, 1,024, 2,016	1
A10G	Qwen 1.5B	U	32	16, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	1
A10G	Qwen 1.5B	U	256	16, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	1
A10G	Qwen 3B	R	32	64, 256, 512, 1,024, 32,736	1
A10G	phi-2	R	32	64, 256, 512, 1,024, 2,016	1
A10G	Llama 3.2-3B	R	32	64, 256, 512, 1,024	1
H100	Qwen 1.5B	U	32	1,024, 16,384	1
L4	pythia-1.4b	R	32	64, 256, 512, 1,024, 2,016	1
L4	Qwen 1.5B	U	32	16, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	2
L4	Qwen 1.5B	U	256	16, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	2
L4	Qwen 3B	R	32	64, 256, 512, 1,024, 32,736	1
L4	phi-2	R	32	64, 256, 512, 1,024, 2,016	1
L4	Llama 3.2-3B	R	32	64, 256, 512, 1,024	1
Pilot	Qwen 1.5B	R	32	512, 2,048, 8,192	1
Pilot	Qwen 1.5B	R	256	512, 2,048, 8,192	1

B Per-board frozen scorecards

The release committed the Ampere predictions under its API tier name before the sweeps. The presentation below separates observed A10 from A10G without changing those predictions. “Controls” cover the repeated-text auxiliary models; main sessions use UltraChat. Anchor, interpolation and extrapolation refer to the calibration length range, not observed bandwidth precision. Error includes baseline conditions; correct/resolved counts concern the resident hit and reload decisions.

Table 3: Frozen scores by observed board and calibration region or session. Error is median absolute TTFT error in milliseconds, over all complete predicted conditions. Decisions exclude unresolved paired intervals.

Board/link	Scope	Correct/resolved	Unresolved	Cells	Error
L4 x16	all	574 / 636	13	767	3.203
L4 x16	anchor	246 / 248	5	299	2.822
L4 x16	interpolate	231 / 289	8	351	2.981
L4 x16	extrapolate	97 / 99	0	117	4.007
L4 x16	l4-controls	193 / 201	8	247	1.506
L4 x16	l4-main-1	188 / 216	4	260	3.681
L4 x16	l4-main-2	193 / 219	1	260	3.654
A10 x16	all	192 / 206	14	260	2.040
A10 x16	anchor	59 / 63	3	78	2.275
A10 x16	interpolate	96 / 105	5	130	2.308
A10 x16	extrapolate	37 / 38	6	52	1.341
A10 x16	a10-main-1	192 / 206	14	260	2.040
A10G x8	all	398 / 426	3	507	5.758
A10G x8	anchor	178 / 187	0	221	5.094
A10G x8	interpolate	170 / 185	2	221	5.511
A10G x8	extrapolate	50 / 54	1	65	8.371
A10G x8	a10-controls	193 / 209	0	247	5.031
A10G x8	a10-main-2	205 / 217	3	260	6.433
H100 x16	all	20 / 21	1	26	6.579
H100 x16	anchor	11 / 11	0	13	0.332
H100 x16	extrapolate	9 / 10	1	13	9.999
H100 x16	h100-main	20 / 21	1	26	6.579

C Reproduction without new experiments

The archive is checksum-verified, and the claim inventory and figure sources are part of `v0.1.0-data`. The following commands reconstruct the released analysis on CPU. They do not authorize or launch GPU work. The manuscript includes released figure PDFs, with only the throughput figure reflowed for print. Its number registry maps empirical values to release files, selectors, and claim numbers; `NUMBERS.md` lists all uses.

```
git clone https://github.com/mohi-devhub/reload-or-recompute.git
cd reload-or-recompute
git checkout v0.1.0-data
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r requirements-analysis.txt
python3 scripts/materialize_results.py
python3 scripts/analyze_phase2.py --from-results outputs/results.jsonl
python3 scripts/report_phase2.py
python3 scripts/analyze_phase3.py --from-results outputs/results.jsonl
python3 scripts/audit_phase3.py
python3 scripts/report_phase3.py
python3 scripts/consolidate_study.py --check --figures
```

The manuscript is a later local draft over that data tag. From its `paper/` directory, standard compilation is:

```
pdflatex -interaction=nonstopmode -halt-on-error main.tex
bibtex main
pdflatex -interaction=nonstopmode -halt-on-error main.tex
pdflatex -interaction=nonstopmode -halt-on-error main.tex
```

Citation verification records distinguish publication metadata from experiment measurements. New source checks correct bibliography metadata only; they add no experimental results. The author-motivation paragraph in the introduction remains an explicit placeholder for author revision.